SQL_ID / SID Diagnostic Script

README: a plain-language guide to **sqlid_diagnostic_v1.2.8.3.sql** and its optional e-mail helper **sqlid_mail.py**

Document purpose	Explain what the script does, what each part of the generated HTML report shows, where the data comes from, and how to read it.
Audience	DBAs and technical users who may not know every Oracle view, SQL*Plus setting or PL/SQL feature the script uses.
Primary source	The SQL file itself. Where a comment in the file and the executable code disagree, this README says so (chapter L) instead of choosing silently.
Version labels in the source	File name: v1.2.8.3. Banner inside the file: 'version 2 (Oracle 19c; 12c+ compatible)'. Change notes: V1.2, V1.2.7, V1.2.8 and V2. See chapter C and chapter L.
Author line in the source	'organised by :- Biswajit Datta'
Prepared	11 October 2026

Safety notice. This README is documentation. It contains no statements that change a database. Saving a guide as a PDF does **not** make the SQL script safe to run: the script's safety depends on its own safeguards (chapter E), on the account you run it with, and on you reviewing the file first.

Recommendations are guidance only. Where the generated report prints a recommended action (for example gathering statistics, rebuilding an index, or loading a plan baseline), the script only *prints it as text*. It never executes it.

Contents

A. Purpose, scope and compatibility 4

A.1 What the script is for . . . 4

A.2 What the script is not . . . 4

A.3 Scope and compatibility stated by the source . . . 4

A.4 How to read this README . . . 4

B. Key terms and abbreviations 5

C. Version information and change notes 6

C.1 Version labels found in the source . . . 6

C.2 Change notes, summarised in the order they appear in the file . . . 6

D. Configuration switches and SQL*Plus settings 7

D.1 The DEFINE switches . . . 7

D.2 Thresholds that are not configurable . . . 7

D.3 SQL*Plus session settings the script applies . . . 8

E. Read-only safeguards and their limits 8

E.1 What the source says it does . . . 8

E.2 The privilege check . . . 8

E.3 Optional e-mail . . . 9

E.4 Limits of these safeguards . . . 9

F. How to run the script 9

F.1 Preparation . . . 9

F.2 Invocation . . . 9

F.3 Input accepted . . . 9

F.4 Output and file naming . . . 10

G. Privileges and licensing 10

G.1 Access the header lists as required . . . 10

G.2 Additional, optional access named in the change notes . . . 10

G.3 Objects the code reads that are not in those lists . . . 11

G.4 Licensing . . . 11

G.5 What happens when access is missing . . . 11

H. Live, cursor-cache and historical data 11

Report Opening Block (before Section 1) 13

Section 1: Current Execution Status, SQL Monitor and Execution Plan 14

1.1 Execution status . . . 14

1.2 SQL text . . . 14
1.3 Sessions currently running or last run this SQL_ID . . . 14
1.4 Real-time SQL Monitor summary (10 most recent executions) . . . 14
1.5 SQL Monitor report (text) . . . 15
1.6 Execution plan from the cursor cache (with actual statistics when available) . . . 15
1.7 Execution plan from AWR (most recent plan_hash_value) . . . 16

Section 2: New SQL_ID Check and Force Matching Signature 17

2.1 History check . . . 17
2.2 SQL_IDs sharing the same FORCE_MATCHING_SIGNATURE . . . 17

Section 3: Execution History - Plan Hash, Average Elapsed, Average Logical I/O 17

3.1 AWR history grouped by plan (shows plan flips, if any) . . . 17
3.2 Current in-memory cursor statistics (V$SQL) . . . 18

Section 4: Blocking and Wait Event Analysis 19

4.1 Current waits for sessions running this SQL_ID . . . 19
4.2 Blocking chain: who is blocking, and what are they doing . . . 19
4.3 Wait-class resolution guidance (for waits happening right now) . . . 19
4.4 Historical Wait Profile - Last 7 Days . . . 20

Section 5: Plan Stability Check 21

5.1 Result . . . 21
5.2 Key facts . . . 21
5.3 Plan history and execution performance by plan . . . 21
5.4 Correlation with performance and environment . . . 21

Section 6: Findings and Recommendations 24

6.1 Measured workload of the analysed plan . . . 24
6.2 Findings and recommendations . . . 24

I. How to read the findings 25

I.1 Severity labels . . . 25
I.2 Parts of a finding . . . 25
I.3 Findings catalogue and thresholds . . . 25
I.4 Why a finding appears or does not . . . 27

J. Privacy and sharing cautions 27

K. Troubleshooting and interpretation notes 27

Other interpretation points . . . 28

L. Source notes and discrepancies 28

A. Purpose, scope and compatibility

A.1 What the script is for

The script takes one input, either a **SQL_ID** (the 13-character identifier Oracle gives a SQL statement) or a **SID** (the session identifier of a connected database session), and writes a single self-contained HTML report about that statement. It is meant for investigating why a statement is slow, why its behaviour changed, whether something is blocking it, and whether the objects it uses have healthy optimizer statistics.

The report looks at the statement from several angles. It reads the **cursor cache** (the shared memory where Oracle keeps parsed statements and their run-time counters), the **AWR** and **ASH** history (periodic snapshots and sampled session activity stored by Oracle's repository, see chapter B), the statement's **execution plan** and **plan hash value** (a number that identifies the plan's shape), its **bind variables** (placeholders whose values are supplied at run time), its work counters such as **buffer gets** (block reads from memory) and **physical reads** (block reads from disk), and its **wait events** (what a session was waiting for). Chapter B defines every term in more detail.

A.2 What the script is not

- It is not a tuning tool that changes anything. It reads data and writes an HTML file (plus, optionally, an e-mail).
- It does not run the actions it recommends. DBMS_STATS, DBMS_SPM and ALTER INDEX style actions appear only as text inside findings.
- It is not proof of a root cause. Findings are drawn from evidence in the views it can read and from fixed thresholds; they are starting points for a DBA's judgement.
- It cannot report on data that has aged out of memory or AWR, or on views the running account is not allowed to read.

A.3 Scope and compatibility stated by the source

- The banner states: 'Oracle 19c; 12c+ compatible'. The source does not list the exact releases it was tested on.
- Section 1.4's code comment says the V$SQL_MONITOR columns it uses exist 'from 11.2 onwards'. The script also checks at run time whether certain columns exist in V$SQL_PLAN_MONITOR and in the optimizer-statistics views, and substitutes empty values when they do not.
- Some printed advice mentions 19c-specific syntax (for example an online table move), so advice text may not suit every 12c system.
- The script is written for SQL*Plus. It uses SQL*Plus commands (DEFINE, ACCEPT, SPOOL, HOST) that other clients may not support.

A.4 How to read this README

Each report subsection is described in a small card with the same rows: **Shows** (what it displays), **Data tier** (live, cursor cache, AWR/ASH history, or data dictionary; see chapter H), **Source** (the Oracle views or packages queried), **How to read it** and **Limits** (missing privileges, licensing, aged-out data). Report headings in this README use the same numbers and titles as the SQL file.

Source-based statements describe what the code does. Statements marked **Reader note** are general Oracle knowledge added for clarity and are not claims made by the script.

B. Key terms and abbreviations

Term	Meaning
SQL_ID	A 13-character identifier Oracle derives from the text of a SQL statement. The same text always gives the same SQL_ID.
SID, SERIAL#	SID identifies a database session; SID plus SERIAL# identifies it uniquely over time, because a SID can be reused after a session ends.
Cursor, child cursor	The parsed, executable form of a statement held in memory. One SQL_ID can have several child cursors, for example because sessions had different optimizer settings or bind values.
Cursor cache	The part of the shared pool (library cache) that holds cursors and their run-time statistics. Read through VSQL, VSQLSTATS, V$SQL_PLAN and similar views. Entries age out and are lost on restart.
Execution plan	The step-by-step method the optimizer chose to run the statement (which tables to read, in which order, with which indexes and joins).
Plan hash value	A number computed from the shape of an execution plan. Different values for the same SQL_ID mean the optimizer has produced different plans.
FORCE_MATCHING_SIGNATURE	A number computed from the statement text after literal values are replaced by placeholders. Statements that differ only in literals share it. The script treats 0 as 'not applicable'.
Bind variable; peeked bind	A placeholder (such as :1 or :name) whose value is supplied at execution. When the optimizer first parses a statement it may 'peek' at the bind values to choose a plan; these are the peeked binds.
Buffer gets (logical reads)	How many times a block was read from the buffer cache in memory. High buffer gets mean the statement touches many blocks.
Physical reads (disk reads)	Blocks that had to be read from storage because they were not in the buffer cache.
Wait event, wait class	When a session cannot continue it records what it waits for (a disk read, a lock, a latch). Events are grouped into classes such as User I/O, Concurrency, Application, Cluster, Network, Commit and Idle.
Blocking session	A session holding something (typically a row lock) that another session needs.
AWR (Automatic Workload Repository)	Oracle's repository of periodic performance snapshots, read through DBA_HIST_* views. Covers days of history. Requires the Diagnostics Pack licence (per the script's header).
Snapshot	One point-in-time capture stored in AWR. AWR 'delta' columns hold the change between two snapshots.
ASH (Active Session History)	Samples of what active sessions are doing. The script states AWR keeps one of every ten one-second ASH samples, so one AWR sample stands for about 10 seconds of active time.
SQL Monitor	A Tuning Pack feature that records detailed run-time information for statements that are long running, parallel or hinted to be monitored. Read through V$SQL_MONITOR and DBMS_SQLTUNE.
Optimizer statistics	Facts about tables, columns and indexes (row counts, blocks, distinct values) the optimizer uses to estimate costs. Gathered with DBMS_STATS.
Stale statistics	Statistics Oracle flags (STALE_STATS = YES) because enough data changed since they were gathered. Staleness alone does not prove a performance problem.
Histogram	Extra column statistics that describe uneven (skewed) value distributions.
Cardinality, E-Rows, A-Rows	The number of rows a plan step returns: E-Rows is the optimizer's estimate; A-Rows is the actual count measured during execution.
Clustering factor	An index statistic describing how well the table's row order matches the index order. A value near the number of rows means consecutive index entries point to different table blocks.
Full table scan	Reading every block of a table instead of using an index.
SQL plan baseline, SQL profile, SQL patch	Oracle mechanisms that influence or pin the plan used for a statement. The script reports whether child cursors use them.
Adaptive cursor sharing (bind sensitive / bind aware)	Oracle may create different child cursors for different bind values when a statement's best plan depends on them.

Term	Meaning
Work area, PGA, one-pass, multi-pass	Memory a sort or hash operation uses (taken from the PGA). If it is too small the operation spills to temporary disk: one pass over temp, or several passes (more costly).
Segment, extent, block	Physical storage units. A segment is the storage of a table or index; it consists of extents, which consist of blocks.
Chained / migrated rows	Rows that do not fit in one block and need an extra block read to fetch. Counted in CHAIN_CNT, which only the ANALYZE command populates.
Index-organized table (IOT)	A table whose data is stored in its primary-key index.
Diagnostics Pack, Tuning Pack	Separately licensed Oracle options. The script's header says AWR, ASH and DBMS_SQLTUNE need them.
Dynamic SQL	SQL assembled as text at run time (EXECUTE IMMEDIATE, OPEN ... FOR). The script uses it so that a missing privilege only skips one subsection.
Anonymous PL/SQL block	A block of PL/SQL code (SQL command type 47) with no stored name. It has no execution plan of its own; the SQL it runs has its own SQL_IDs.
Substitution variable, bind variable (SQL*Plus)	A substitution variable (&name or &&name) is replaced in the text before SQL*Plus runs a line. A SQL*Plus bind variable (VARIABLE / EXEC) passes a value into a PL/SQL block.
V$, GV$, DBA_, ALL_ views	V$ views show the connected instance only; GV$ views show all RAC instances. DBA_ views show every object (needs privilege); ALL_ views show objects the user can access.
RAC	Real Application Clusters: several instances sharing one database.
SPOOL, DBMS_OUTPUT	SPOOL writes SQL*Plus output to a file. The script builds the HTML with DBMS_OUTPUT (text lines) and spools it to the report file.

C. Version information and change notes

C.1 Version labels found in the source

- File name: **sqlid_diagnostic_v1.2.8.3.sql**.
- Banner comment: **'SQL_ID / SID DIAGNOSTIC SCRIPT - version 2 (Oracle 19c; 12c+ compatible)'**.
- The header's run instruction names **sqlid_diagnostic_v2.sql**, and the mail helper's own comment refers to the same name.
- The change notes use the labels FIX (section 1.4), V1.2.8, V1.2.7, V2 and V1.2 (twice). There is no change note for a '.3' revision.

C.2 Change notes, summarised in the order they appear in the file

Label in file	What the file says changed
FIX (section 1.4)	Corrected ORA-00904 on 'PGA_ALLOCATED' (V$SQL_MONITOR has no such column). The query now uses valid V$SQL_MONITOR columns; rows processed and work-area memory come from V$SQL_PLAN_MONITOR (column-checked per release). Missing pack, missing privilege and no-data cases give an informational message.
V1.2.8 (sections 1.5, 1.6)	1.5 gained two bind subsections: 1.5.1 bind values used by the monitored executions (V$SQL_MONITOR.BINDS_XML) and 1.5.2 binds captured by the optimizer (V$SQL_BIND_CAPTURE). 1.6 also prints peeked binds. New switch show_binds (default Y). Optional extra access: V$SQL_MONITOR (BINDS_XML, Tuning Pack) and V$SQL_BIND_CAPTURE.
V1.2.7 (sections 1.6, 1.7)	Plans are shown as three collapsible blocks: plan table, plan outline (hints that reproduce the plan) and predicate information / notes. 1.7 now requests 'ALL +OUTLINE'. New helper procedure PUT_XPLAN.
V2	The script writes the HTML report directly (no raw SQL*Plus spool and no sqlid_report.py). The whole report is one PL/SQL block (fixes SP2-0734 and PLS-00201 for C_WAITS / V_FOUND). All V$ / DBA_HIST queries are dynamic SQL with bind variables. Section 2.2 corrected (V$SQLSTATS has no FIRST_LOAD_TIME; it comes from V$SQLAREA). Section 4 redesigned. Section 5 (Plan Stability Check) added. Section 6 Findings and Recommendations.

Label in file	What the file says changed
V1.2 (section 5.4)	Related table and index statistics, and fragmentation analysis, added: STALE / NOT STALE / UNKNOWN / MISSING status, index validity, partition counts, and a statistics-based fragmentation indicator. No VALIDATE STRUCTURE or segment analysis is run; HIGH is never assigned automatically. Optional DBA_* access listed; ALL_* views are used as a fallback.
V1.2 (section 6)	Section 6 shows measured workload first (6.1), then findings (6.2) carrying evidence from this SQL_ID, a recommended action and how to confirm the result; findings are sorted High / Review / Info with a named 'Main finding'. A finding appears only when its evidence crosses a stated threshold.
READ-ONLY GUARANTEES	Added 'after a security review': read-only transaction, SELECT-only statements, a privilege check, e-mail off unless send_mail = Y. See chapter E.

The notes are not in version order (the FIX note sits above V1.2.8, and V1.2.x notes sit around a note labelled V2), and the labels do not match the file name. This README reports the labels as written and does not decide which is newest. See chapter L.

D. Configuration switches and SQL*Plus settings

D.1 The DEFINE switches

A **DEFINE** near the top of the file sets a SQL*Plus substitution variable. Change a value by editing the line before running the script. The comments in the file describe defaults; the table below compares them with the value actually assigned in the file.

Switch	Value assigned in the file	Default the comments state	What it controls
allow_privileged	**Y**	N	N: the report stops if the account holds privileges that could change data, objects or the instance. Y: the report continues. See chapter E.
send_mail	**Y**	N	N: the report is only written to a file. Y: after the report is written, the script calls the e-mail helper (an operating-system command).
show_binds	Y	Y	Y: show bind values in 1.5.1, 1.5.2 and peeked binds in 1.6. N: leave them out (recommended before e-mailing or sharing; see chapter J).
rpt_python	'python3'	(none stated)	Interpreter used to run the e-mail helper.
rpt_helper	'sqlid_mail.py'	(none stated)	Name of the e-mail helper script.
frag_min_mb	100	(none stated)	Segments smaller than this many MB are not flagged in the fragmentation indicators.
frag_density	50	(none stated)	Tables: estimated block density (%) below which the indicator shows MODERATE / REVIEW.
frag_ratio	2	(none stated)	Indexes: allocated blocks divided by leaf blocks above which the indicator shows MODERATE / REVIEW.

Discrepancy: as shipped, allow_privileged and send_mail are **Y**, while the comments describe N as the default for both. That means, with the file as supplied, the DBA-privilege check does not stop the report, and the e-mail helper runs (and prompts for an address) after every report. Check these two lines before running. See chapter L.

D.2 Thresholds that are not configurable

Some thresholds are fixed in the code and cannot be changed with a DEFINE. Examples: a row-estimate error of 10 times or more (constant c_misest) in the object findings, and the thresholds in chapter I (for example 40% of elapsed time in user I/O). To change them you would have to edit the PL/SQL, which is outside the scope of this README.

D.3 SQL*Plus session settings the script applies

Setting	Plain-language meaning
SET DEFINE ON / OFF	ON lets SQL*Plus replace &names with values. The script turns DEFINE OFF while writing the HTML, because the HTML contains literal ampersand characters.
SET SERVEROUTPUT ON SIZE UNLIMITED FORMAT WRAPPED	Allows the PL/SQL block to print text lines (DBMS_OUTPUT) of any total size; the report text goes through this.
SET VERIFY OFF, ECHO OFF, FEEDBACK OFF, TIMING OFF	Suppress SQL*Plus chatter (old/new substitution lines, echoed commands, row-count feedback, timing).
SET HEADING, PAGESIZE, LINESIZE 32767, LONG, LONGCHUNKSIZE, TRIMSPOOL, TAB OFF, SQLBLANKLINES ON	Output layout: no page breaks or headings while spooling, very wide lines, large LOB display (1,000,000), no trailing blanks or tabs, and blank lines allowed inside statements.
CLEAR SCREEN	Clears the terminal at the start.
SET TERMOUT OFF / ON	Hides or shows output on the screen. The report is written to the spool file, not the screen.
SET TRANSACTION READ ONLY	Starts a read-only transaction so the database refuses data changes in this session (ORA-01456). See chapter E.
ACCEPT input_val	Prompts for the SQL_ID or SID.
VARIABLE / EXEC	Create SQL*Plus bind variables and give them values (SQL_ID, SID, input text, the thresholds, show_binds, the privilege-check result) so the PL/SQL block needs no substitution variables.
SPOOL file / SPOOL OFF	Writes everything printed in between to the report file.

E. Read-only safeguards and their limits

E.1 What the source says it does

- **Read-only transaction.** The session issues SET TRANSACTION READ ONLY before any query, so the database itself refuses any data change from this session (ORA-01456).
- **SELECT-only.** Every dynamic statement is a SELECT. DBMS_XPLAN and DBMS_SQLTUNE.REPORT_SQL_MONITOR are called from a SELECT. According to the header, no INSERT, UPDATE, DELETE, MERGE, DDL, COMMIT or DBMS_STATS call is executed.
- **Advice is text.** DBMS_STATS, DBMS_SPM and ALTER INDEX style statements in section 6 and in the 5.4 findings are printed in the report, never run.
- **Privilege check.** Before generating the report the script looks at the account's session privileges (see E.2).
- **E-mail is optional.** It is an operating-system call, not a database operation, and only happens when send_mail = Y (see E.3).

E.2 The privilege check

The script reads the account's session privileges and collects any of these 20: SYSDBA, SYSOPER, ALTER SYSTEM, ALTER DATABASE, CREATE ANY TABLE, ALTER ANY TABLE, DROP ANY TABLE, INSERT ANY TABLE, UPDATE ANY TABLE, DELETE ANY TABLE, CREATE ANY INDEX, ALTER ANY INDEX, DROP ANY INDEX, CREATE ANY PROCEDURE, EXECUTE ANY PROCEDURE, ANALYZE ANY, GRANT ANY PRIVILEGE, GRANT ANY ROLE, ALTER USER, DROP USER.

- If none are found, the report runs normally.
- If one or more are found and **allow_privileged is not Y**, the HTML file contains only the opening block and a red 'Report not generated' message listing the privileges.
- If one or more are found and **allow_privileged = Y**, the report continues. The file's comment says 'a warning is shown'; in the code reviewed, no warning is printed in this case (chapter L).
- If the privilege check itself fails, the result is stored as an error message, which counts as 'privileges found'; with allow_privileged other than Y the report stops.

E.3 Optional e-mail

- After the report is written, if send_mail = Y, the script creates a one-line temporary SQL*Plus script that runs the helper (python3 sqlid_mail.py with the report name, SQL_ID and database name), runs it, then deletes the temporary file with an operating-system command (rm -f).
- The helper asks for an address on the terminal. A blank answer sends nothing. Up to three attempts are allowed for an invalid address.
- It sends the report as an HTML body *and* as an attachment. Whatever is in the report (including bind values if show_binds = Y) is therefore in the e-mail.
- SMTP settings come from environment variables: SMTP_HOST (default localhost), SMTP_PORT (25), SMTP_SECURITY (none, starttls or ssl), SMTP_USER, SMTP_PASSWORD, SMTP_FROM.
- A failed send is reported on screen and does not break the SQL*Plus session.

E.4 Limits of these safeguards

- **Stated by the source:** the SQL_ID or SID you type is substituted into SQL*Plus text (a SQL*Plus limitation). It can only run with the privileges of the read-only account.
- **Reader note:** the read-only transaction governs database changes. It does not govern what the script does outside the database: creating the HTML file, creating and deleting a temporary file, running operating-system commands (HOST) and sending e-mail.
- **Reader note:** the privilege check is a fixed list. It cannot detect every way an account could be able to change data (for example object-level grants on application tables).
- **Reader note:** reading dictionary and AWR/ASH views is not free of cost on a busy system. 'Read-only' does not mean 'zero load'.
- **Best practice stated by the file:** use a dedicated account with SELECT grants only.
- **Reading this PDF is safe; running the script is a separate decision.** Review the SQL file, the e-mail helper and your settings first.

F. How to run the script

F.1 Preparation

- SQL*Plus client with a working connection to the database.
- Run the script as a **script file**. Do not paste it line by line into an interactive prompt (stated in the header).
- If the file was edited on Windows, convert line endings first (the header suggests dos2unix).
- The current directory must be writable: the report file (and, with e-mail, a temporary script) is created there.
- For e-mail: python3 and sqlid_mail.py in the same directory, SMTP environment variables set, and an operating system where the command rm -f exists (the script deletes its temporary file with it).
- A dedicated account holding only the SELECT grants of chapter G.

F.2 Invocation

Start SQL*Plus and connect with the dedicated account, then run the script file by name with the @ command: `@sqlid_diagnostic_v1.2.8.3.sql`. The header's own example uses a different file name (chapter L); use the name of the file you actually have.

F.3 Input accepted

The script prompts: `Enter SQL_ID or SID (session id) to diagnose:`. It then classifies the answer:

You enter	What the script does	Result
Digits only (for example a SID)	Looks the session up in V$SESSION and uses its current SQL_ID, or its previous SQL_ID if the session is idle.	SQL_ID resolved; SID shown as given. If the session or its SQL_ID does not exist: 'UNKNOWN'.

You enter	What the script does	Result
Exactly 13 letters/digits	Treats it as a SQL_ID (converted to lower case). Looks for a session that has it as current or previous statement and shows the highest SID found.	SQL_ID resolved; 'Related SID' may be N/A.
Anything else	Rejected.	'INVALID': the HTML explains the input is neither a numeric SID nor a 13-character SQL_ID.

Reader note: the digits-only test is applied first. A 13-character SQL_ID that happens to consist only of digits would be read as a SID. This is rare but possible.

F.4 Output and file naming

- File name: `sqlid_diag_<sql_id>_<YYYYMMDD_HH24MISS>.html` in the current directory. For rejected or unresolved input the SQL_ID part reads INVALID or UNKNOWN.
- On screen you see 'Generating HTML report for SQL_ID ...' and, at the end, 'HTML report written to: ...'.
- The HTML is self-contained (embedded styling, no external files), prints cleanly, and uses collapsible blocks for long text such as plans.
- The whole report is one PL/SQL block, so each subsection runs inside its own error handler: when something fails you see a yellow note naming the subsection and the Oracle error, and the rest of the report continues.

G. Privileges and licensing

G.1 Access the header lists as required

The header says the grants must be **direct** (not via a role) 'because of PL/SQL'. This README repeats that statement from the source and has not tested it.

Kind	Objects named in the header
SELECT on dynamic performance views	V$SESSION, V$SQL, V$SQLAREA, V$SQLSTATS, V$SQL_MONITOR, V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL, VSQL_SHARED_CURSOR, VSQL_WORKAREA, V$INSTANCE, V$DATABASE
SELECT on data dictionary / AWR views	DBA_TAB_STATISTICS, DBA_HIST_SQLSTAT, DBA_HIST_SQLTEXT, DBA_HIST_SQL_PLAN, DBA_HIST_SNAPSHOT, DBA_HIST_ACTIVE_SESS_HISTORY, DBA_HIST_PARAMETER
EXECUTE on packages	DBMS_XPLAN, DBMS_SQLTUNE

G.2 Additional, optional access named in the change notes

Needed for	Objects
Related table / index statistics and fragmentation (5.4)	DBA_TAB_STATISTICS, DBA_IND_STATISTICS, DBA_TABLES, DBA_INDEXES, DBA_IND_PARTITIONS, DBA_IND_SUBPARTITIONS, DBA_SEGMENTS, DBA_OBJECTS, DBA_TAB_MODIFICATIONS. Without DBA_* the ALL_* views are used (only objects visible to the user). Without DBA_SEGMENTS fragmentation is NOT ASSESSED.
Findings (section 6)	V$SQLSTATS, V$SQL, V$SQL_WORKAREA, V$SQL_PLAN_STATISTICS_ALL, V$PARAMETER, DBA_HIST_SNAPSHOT, DBA_HIST_ACTIVE_SESS_HISTORY, DBA_IND_COLUMNS (or ALL_IND_COLUMNS). 'A missing one only skips its finding.'
Bind information (1.5.1, 1.5.2)	V$SQL_MONITOR (BINDS_XML, Tuning Pack) and V$SQL_BIND_CAPTURE. 'A missing one only skips that subsection.'

G.3 Objects the code reads that are not in those lists

The code or its messages also refer to V$SQL_PLAN_MONITOR (section 1.4), V$ACTIVE_SESSION_HISTORY (the PL/SQL-block path of section 6), SESSION_PRIVS (privilege check) and ALL_TAB_COLUMNS (checks which columns exist in this release). A missing grant on these affects only the part that uses them.

G.4 Licensing

- The header states that AWR, ASH and DBMS_SQLTUNE need the **Diagnostics Pack and Tuning Pack** licence. V$SQL_MONITOR is described as Tuning Pack.
- Only section 1.4 checks the setting CONTROL_MANAGEMENT_PACK_ACCESS. If the Tuning Pack is not enabled there, 1.4 shows an informational message and queries nothing.
- Other pack-dependent parts (AWR history, ASH, the 1.5 SQL Monitor report) are attempted whenever the account has the privilege. The script does not check your licence for them. This README gives no licensing advice; confirm entitlement with your licensing contact before running it.

G.5 What happens when access is missing

Situation	Effect
AWR views DBA_HIST_SQLSTAT or DBA_HIST_SNAPSHOT not readable	A yellow warning at the top. All AWR/ASH-based parts are limited to what is still in memory (1.7, 3.1, 4.4 and several others say 'AWR is not accessible').
A view or package not readable	Only that subsection shows an informational or warning note. The rest of the report continues.
DBA_* statistics views not readable	ALL_* views are used; objects owned by other schemas may show as UNKNOWN.
DBA_SEGMENTS not readable	Fragmentation indicators show NOT ASSESSED.
Neither DBA_* nor ALL_* statistics views readable	Related objects are listed by name only.

H. Live, cursor-cache and historical data

The report mixes four kinds of information. Knowing which one you are reading tells you how current it is and how long it lasts.

Data tier	What it is	Views used (examples)	How to read it
LIVE	State of sessions right now on the connected instance.	V$SESSION; V$SQL_MONITOR while a statement is EXECUTING	Valid only at the moment the report ran. An empty result means nothing is happening now, not that nothing ever happened.
CACHE	Statistics and plans for cursors still in the shared pool (and monitor entries still in memory). Cumulative since the cursor was loaded.	VSQL, VSQLSTATS, V$SQLAREA, V$SQL_PLAN, V$SQL_PLAN_STATISTICS_ALL, V$SQL_WORKAREA, V$SQL_BIND_CAPTURE, VSQL_SHARED_CURSOR, VSQL_MONITOR, V$SQL_PLAN_MONITOR	Lost when the cursor ages out or the instance restarts. Counters can include old executions.
AWR / ASH	History kept in the repository. Rates and averages come from snapshot deltas.	DBA_HIST_SQLSTAT, DBA_HIST_SNAPSHOT, DBA_HIST_SQLTEXT, DBA_HIST_SQL_PLAN, DBA_HIST_ACTIVE_SESS_HISTORY, DBA_HIST_PARAMETER	Needs the Diagnostics Pack. Covers days, but is sampled: ASH history keeps one in ten samples; the script's time estimate is samples x 10 seconds.

Data tier	What it is	Views used (examples)	How to read it
DICTIONARY	Current metadata about tables and indexes.	DBA_TAB_STATISTICS, DBA_IND_STATISTICS, DBA_TABLES, DBA_INDEXES, DBA_SEGMENTS, DBA_OBJECTS, DBA_TAB_MODIFICATIONS, DBA_IND_COLUMNS	Describes the objects as they are now, even when the plan being analysed is older.

Reader note: all V$ views reflect the instance you are connected to. In a RAC database, sessions and cursors on other instances are not shown (the script itself says so for blockers on another instance in 4.2).

Report Opening Block (before Section 1)

Before section 1, the HTML shows a banner (SQL_ID, database name, generation time) and a table of facts, then may show messages that explain why the rest is limited or missing.

Shows	• **Input provided**: exactly what you typed. • **Resolved SQL_ID** and **Related SID**. • **Database / Instance** with Oracle version, **Host** and **Report run by** (the session user).
Data tier	LIVE (V$SESSION, V$INSTANCE, session context).
Early messages	• **Report not generated** (red): the privilege check stopped the report (chapter E). • **Input is neither a numeric SID nor a 13-character SQL_ID**, or **No SQL_ID could be resolved**: the report ends here. • **AWR views are not readable** (yellow): history parts will be limited. • **This SQL_ID is an anonymous PL/SQL block** (yellow): plan-based checks do not apply; section 6 instead lists statements the block ran. Run the script again for those SQL_IDs.
Limits	A SID that is idle shows its previous SQL_ID, which may not be the statement you care about. A SID that has ended or changed since you looked can resolve to a different statement.

Section 1: Current Execution Status, SQL Monitor and Execution Plan

Section 1 answers: is it running, what is the statement, who is running it, how did recent executions behave, which binds were used, and which plan is it using? Subsections 1.1, 1.3 and 1.4 are live or near-live; 1.2, 1.5 and 1.6 are cursor-cache; 1.7 is AWR.

1.1 Execution status

Shows	Whether this SQL_ID is executing now: a badge RUNNING / IN PROGRESS (statement executing) or NOT RUNNING, plus the number of active sessions executing it.
Data tier	LIVE
Source	V$SESSION (SQL_ID with status ACTIVE) and V$SQL_MONITOR (status EXECUTING).
How to read it	• RUNNING means at least one active session or an executing monitor entry exists. • NOT RUNNING does not mean the statement is healthy; it only says it is not running at this moment.
Limits	If V$SQL_MONITOR cannot be read, that part is treated as zero without a visible message. Only the connected instance is checked.

1.2 SQL text

Shows	The full statement text in a formatted block, with long lines wrapped, up to 400 lines.
Data tier	CACHE first, AWR as fallback.
Source	V$SQLSTATS.SQL_FULLTEXT; if absent, DBA_HIST_SQLTEXT.SQL_TEXT.
How to read it	Check that this is the statement you meant. Literal values (if any) appear here as written.
Limits	If the text is in neither the shared pool nor AWR you get an information note.

1.3 Sessions currently running or last run this SQL_ID

Shows	A table: SID, Serial#, User, Session Status, 'This SQL_ID is' (Current statement or Previous statement), SQL Exec Start, 'In Current State For' (time since the session's last call began), Wait Class and Current / Last Wait Event.
Data tier	LIVE
Source	V$SESSION where SQL_ID or PREV_SQL_ID matches. Current statements are listed first.
How to read it	• 'Current statement' = the session is on it now. 'Previous statement' = the session has moved on and may be idle. • For a session that is not waiting, the event shown is only its last completed wait.
Limits	Sessions that have ended are gone. Only the connected instance.

1.4 Real-time SQL Monitor summary (10 most recent executions)

Shows	Up to 10 monitored executions, newest first: SID, SQL_ID, SQL Exec ID, Status, Monitoring Start, SQL Exec Start, Elapsed (s), CPU (s), Buffer Gets, Physical Reads (blocks), Rows Processed, Plan Hash Value, Work Area Max (MB).
Data tier	CACHE (monitor entries live in memory); LIVE for executions in progress.
Source	V$SQL_MONITOR. Rows Processed = output rows of plan line 0 and Work Area Max = sum of the work-area maximum memory over plan lines, both from V$SQL_PLAN_MONITOR when those columns exist in this release (otherwise blank).
How to read it	• Compare elapsed and CPU: a large gap points to waiting (I/O, locks) rather than computing. • Compare plan hash values between rows: a change means the plan differed between executions. • Work Area Max shows memory used by sorts/hashes; compare with the work-area finding in section 6.

Limits	• The script first reads CONTROL_MANAGEMENT_PACK_ACCESS. If the Tuning Pack is not enabled it shows an information note and does not query V$SQL_MONITOR. • 'No SQL Monitor entries' is normal for short statements: the script states a statement is monitored only when it runs in parallel, uses the MONITOR hint, or uses more than 5 seconds of CPU or I/O time, and that monitoring needs STATISTICS_LEVEL = TYPICAL or ALL. Entries also age out. • Missing object or privilege errors (ORA-00942, ORA-01031, ORA-01039) are shown as an information note, not an error.

1.5 SQL Monitor report (text)

Shows	A collapsible text report from Oracle's SQL Monitor (report level ALL, text type), up to 3,000 lines. Typically it lists the plan with run-time activity per step for the monitored execution.
Data tier	CACHE
Source	DBMS_SQLTUNE.REPORT_SQL_MONITOR, called from a SELECT. Tuning Pack.
How to read it	Expand 'Show SQL Monitor report'. Use it to find which plan step consumed time, I/O or memory in a monitored run.
Limits	Needs the Tuning Pack and a monitored execution that is still in memory. If the output is shorter than 50 characters the script says no report was returned. Errors are shown as a yellow note.

1.5.1 Bind variables of the monitored executions

Shows	A table: SQL Exec ID, SQL Exec Start, Status, Position, Bind Name, Data Type and Value Used. Execution ID, start and status are printed only on the first row of each execution. Up to 200 rows, newest executions first.
Data tier	CACHE
Source	V$SQL_MONITOR.BINDS_XML (the values the monitored executions actually used), unpacked with XMLTABLE. Tuning Pack.
How to read it	Use these to see which values a slow execution ran with, and to compare fast and slow executions. 'NULL / not recorded' means no value was stored.
Limits	Shown only when show_binds = Y (otherwise an information note explains they are hidden). If no values are recorded (no binds, not monitored, entry aged out) the script says so. Contains application data: see chapter J.

1.5.2 Bind variables captured by the optimizer (V$SQL_BIND_CAPTURE)

Shows	A table: Child #, Position, Bind Name, Data Type, Captured? (YES/NO), Last Captured and Captured Value. Up to 200 rows.
Data tier	CACHE
Source	V$SQL_BIND_CAPTURE.
How to read it	These are **samples**. The script says the optimizer captures binds at hard parse and then about every 15 minutes while the statement runs, so a value can be out of date. A Captured? = NO row shows 'not captured'.
Limits	The script notes some bind types (for example LOBs, long values) or values beyond the capture size limit are not captured. Hidden when show_binds = N (heading 1.5.2 is not printed). Contains application data.

1.6 Execution plan from the cursor cache (with actual statistics when available)

Shows	The plan of the cached cursor as three collapsible blocks: (1) the plan table (open by default); (2) the **plan outline**, the hints that reproduce the plan (the 'Outline Data' section); (3) **predicate information and notes** (open by default). With show_binds = Y the output also lists peeked binds.
Data tier	CACHE
Source	DBMS_XPLAN.DISPLAY_CURSOR with the format 'ALLSTATS LAST +OUTLINE' (plus ' +PEEKED_BINDS' when show_binds = Y).

How to read it	• Read the plan from the innermost (most indented) step outward. Operation names show how each table is accessed (index, full scan) and joined. • With actual statistics, compare E-Rows (estimated) with A-Rows (actual): a gap of 10 times or more is a sign of a cardinality problem (section 6 uses this). • Predicate information shows which conditions are index access conditions and which are filters applied after reading. • The outline can be compared between plans, or used by a DBA when pinning a plan.
Limits	Actual figures (A-Rows, buffers, time) exist only if row-source statistics were collected (STATISTICS_LEVEL = ALL or the GATHER_PLAN_STATISTICS hint, as the script states in section 6). If the cursor is no longer in the shared pool you see 'no longer in the shared pool'. If no outline is stored, a note says so.

1.7 Execution plan from AWR (most recent plan_hash_value)

Shows	The plan with the highest snapshot in AWR for this SQL_ID, labelled 'Most recent AWR plan_hash_value', in the same three collapsible blocks (opened only when no cache plan was shown).
Data tier	AWR
Source	DBA_HIST_SQLSTAT to pick the plan hash value from the latest snapshot, then DBMS_XPLAN.DISPLAY_AWR with the format 'ALL +OUTLINE'.
How to read it	Use it when the cursor has aged out, or to compare with the cached plan. If the two plan hash values differ, the plan changed.
Limits	Needs readable AWR. 'Most recent' means the latest snapshot containing the SQL_ID, which is not necessarily the plan the script treats as 'current' in sections 5 and 6 (chapter L). AWR plans carry no actual statistics.

Section 2: New SQL_ID Check and Force Matching Signature

Section 2 tells you whether this statement is new and whether other SQL_IDs are the same statement with different literal values. The script first finds the statement's FORCE_MATCHING_SIGNATURE (a non-zero value from V$SQLSTATS, falling back to DBA_HIST_SQLSTAT).

2.1 History check

Shows	A table: **Result** (NEW SQL_ID or NOT NEW), number of child cursors in V$SQL, number of AWR rows in DBA_HIST_SQLSTAT (or 'not available'), and the FORCE_MATCHING_SIGNATURE (or an explanation of why there is none).
Data tier	CACHE and AWR.
Source	V$SQL, DBA_HIST_SQLSTAT, V$SQLSTATS.
How to read it	• NEW SQL_ID: no cursor in the cache and no AWR rows. A recently deployed or changed statement looks like this. • NOT NEW: the SQL_ID has been seen before.
Limits	**Reader note:** if AWR is not readable, it is counted as 'no history', so a statement missing from the cache can show as NEW even though AWR might hold it. A signature of 0 or not found means the text is not force-matchable or is in neither V$SQLSTATS nor AWR.

2.2 SQL_IDs sharing the same FORCE_MATCHING_SIGNATURE

Shows	A table of SQL_IDs with the same signature: SQL_ID (the analysed one is badged), signature, distinct plans, executions, First Load Time ('not in cache (aged out)' when unknown) and Last Active Time. Newest first load first.
Data tier	CACHE
Source	V$SQLSTATS joined to V$SQLAREA (first load time comes from V$SQLAREA because V$SQLSTATS has no such column).
How to read it	• Several SQL_IDs here are the same statement with different literals; a literal-heavy application creates many. • A note states how many SQL_IDs share the signature, or that this is the only one.
Limits	V$SQLSTATS only. If the signature exists only in AWR, the note says no matching rows remain in V$SQLSTATS.

Section 3: Execution History - Plan Hash, Average Elapsed, Average Logical I/O

3.1 AWR history grouped by plan (shows plan flips, if any)

Shows	One row per plan hash value: snapshots, executions, average elapsed (s), average buffer gets, average disk reads, first seen and last seen. Newest last-seen first.
Data tier	AWR
Source	DBA_HIST_SQLSTAT joined to DBA_HIST_SNAPSHOT. Averages are the summed deltas divided by summed execution deltas.
How to read it	• Several rows mean the statement has used several plans. Compare average elapsed and buffer gets across them. • A big difference is a plan flip with performance impact; similar numbers suggest the change is harmless.
Limits	Needs AWR. **Reader note:** AWR does not necessarily record every statement in every snapshot, so history can have gaps.

3.2 Current in-memory cursor statistics (V$SQL)

Shows	One row per child cursor: child number, plan hash value, executions, average elapsed (s), average buffer gets, average disk reads, parse calls, first load time, last active time.
Data tier	CACHE
Source	V$SQL.
How to read it	Averages are cumulative since the child was loaded. Different children with different plan hash values show which plans are live now.
Limits	If the SQL_ID is not in the shared pool the script says so.

Section 4: Blocking and Wait Event Analysis

4.1 Current waits for sessions running this SQL_ID

Shows	A headline note (none waiting / some waiting / some blocked) and a table: SID, Serial#, User, Session Status, State, Wait Class, Wait Event, Wait Duration, Blocked?, Blocking SID (with instance and root blocker), Blocking Status and decoded Wait Parameters.
Data tier	LIVE
Source	V$SESSION, with P1/P2/P3 decoded by the script.
How to read it	• State: WAITING = the session is waiting; the 'RUNNING / ON CPU' states = it is working. Event and duration are shown only for WAITING sessions. • Blocking Status: VALID (blocker identified), NO HOLDER, NOT IN WAIT, UNKNOWN, or GLOBAL (blocker is on another RAC instance). • Wait parameters are decoded for enqueue waits (lock type, requested mode, id1/id2) and for cursor / library-cache mutex waits (holder SID, object hash); for latches, Network, Idle and Commit waits no parameters are shown because they carry little diagnostic value.
Limits	Shows nothing if no session is executing this SQL_ID now. Connected instance only.

4.2 Blocking chain: who is blocking, and what are they doing

Shows	Three groups of columns: the blocked session (SID, Serial#, User, waiting on, waiting for), the blocking session (SID with root blocker, Serial#, User, Status ACTIVE or INACTIVE) and the blocker's activity (SQL_ID, current event or explanation, time in state).
Data tier	LIVE
Source	V$SESSION joined to itself on BLOCKING_SESSION.
How to read it	• An **INACTIVE** blocker holding a lock usually has an open (uncommitted) transaction; the script says so in the row. • If the blocker is itself blocked, the row names the next blocker.
Limits	If a session is blocked from another RAC instance, the table is empty and a warning asks you to query GV$SESSION. If nothing is blocked: 'No blocking session detected.'

4.3 Wait-class resolution guidance (for waits happening right now)

For each distinct non-idle wait event currently being waited on by sessions running this SQL_ID, the report prints a generic cause and suggested resolution. These are rules of thumb from the script, not diagnoses:

Wait event pattern	Probable cause (script text)	Suggested resolution (script text, summarised)
db file sequential read	Single-block I/O, typically index lookups.	Check index selectivity and storage latency; consider caching hot blocks.
db file scattered read; direct path read	Multi-block I/O, typically full scans.	Check for missing indexes or stale statistics; consider partitioning or parallelism if the scan is legitimate.
enq: TX	Row-lock contention.	Identify and end the blocking transaction; look for long uncommitted DML or unique-key contention.
enq: TM	DML/DDL table-lock contention.	Check unindexed foreign keys or concurrent DDL.
latch: cache buffers chains	Many sessions reading the same hot blocks.	Reduce logical I/O of the SQL; look for hot blocks; consider partitioning or less concurrency.
library cache / shared pool / cursor: events	Library cache or shared pool contention, often hard parsing of literal SQL.	Use bind variables; review CURSOR_SHARING.
Other latch or mutex	Contention on a shared memory structure.	Identify the structure from the event name and reduce concurrency.
log file sync	Commit-related wait on LGWR.	Check redo I/O latency and commit frequency.
buffer busy waits	Contention on one block, often a hot index leaf block.	Consider reverse-key or hash-partitioned index; verify ASSM.

Wait event pattern	Probable cause (script text)	Suggested resolution (script text, summarised)
gc events	RAC interconnect wait.	Check interconnect; consider service or session affinity.
Network class	Round trips (SQL*Net).	Check client fetch size and network latency.
Concurrency class	Generic concurrency wait.	Review the blocking chain and shorten lock holds.
Anything else	'Wait class X - no specific pattern known to this script.'	Check documentation and AWR.

4.4 Historical Wait Profile - Last 7 Days

Shows	Top 10 wait events (or CPU) for this SQL_ID: Wait Event, Wait Class, Samples, % of Total, a bar, Approx. DB Time, then an 'All other events' row and a Total row.
Data tier	AWR/ASH
Source	DBA_HIST_ACTIVE_SESS_HISTORY, samples of the last 7 days. A sample with no event is labelled 'CPU + Wait for CPU'.
How to read it	Each sample represents roughly 10 seconds of active time, so 'Approx. DB Time' = samples x 10. The top rows show where this statement spends its time.
Limits	Needs AWR and Diagnostics Pack. No samples in 7 days (statement did not run, or history aged out) gives an information note.

Section 5: Plan Stability Check

Section 5 asks whether the statement has used more than one plan and, if so, whether that matters. It first collects every plan hash value seen in the cursor cache and AWR. The **current plan** is the most recently active plan in the cursor cache; if none, the latest plan in AWR; if no timestamps, the first plan listed. Sections 5.4 (object analysis) and 6 use this plan.

5.1 Result

Shows	One verdict note: NO PLAN DATA, STABLE PLAN (one plan hash value observed) or MULTIPLE PLANS / POTENTIAL PLAN INSTABILITY (several).
Data tier	CACHE and AWR.
How to read it	Multiple plans alone do not prove a problem; compare the performance in 5.3. STABLE PLAN only covers the history that is still available.

5.2 Key facts

Shows	• SQL_ID, current plan hash value (and why it was chosen), number of plans, 'Multiple plans?', 'Plan hash value changed?' (with the earlier plan and when the current one was first seen in AWR). • Observed AWR window and snapshot rows, force matching signature, how many SQL_IDs share it, distinct plans across them, and child cursors / distinct optimizer environments.
Data tier	CACHE and AWR.
How to read it	'First seen in AWR' for the current plan approximates when the change happened. Multiple optimizer environments among children means different sessions ran with different optimizer settings.

5.3 Plan history and execution performance by plan

Shows	One row per plan: plan hash value (current is badged), where observed (Cache only, AWR only, Cache and AWR), executions (AWR and cache), average elapsed (s), 'Relative to Fastest Plan' (a multiple) and first / last observed.
Data tier	CACHE and AWR.
How to read it	Executions (AWR) are summed deltas; Executions (cache) are cumulative since load; they overlap and must not be added. Average elapsed uses AWR when available, otherwise the cache.

5.4 Correlation with performance and environment

5.4 is the longest subsection. It starts with short observations and then contains several unnumbered blocks, each introduced by a bold label. The V1.2 change notes call this group 'section 5.4'.

Observations (bullet list)

- **Performance:** flags when the slowest plan's average elapsed time is 2 times or more that of the fastest, or notes the plans are within 2 times of each other.
- **Signature:** how many SQL_IDs share the signature and how many distinct plans they show together.
- **Cursors:** number of child cursors and distinct optimizer environments.

SQL_IDs sharing this FORCE_MATCHING_SIGNATURE

Shown only when more than one SQL_ID shares the signature. Up to 20 most recently active SQL_IDs: Seen In (Cache only / AWR only / Cache and AWR), distinct plans, executions, average elapsed, last active. Source: V$SQLSTATS and DBA_HIST_SQLSTAT.

Child cursors in the shared pool (first 20)

Per child: plan hash value, executions, average elapsed, optimizer environment hash, Bind Sensitive / Bind Aware / Re-optimizable flags, and whether it uses a SQL plan baseline, SQL profile or SQL patch, plus the parsing schema. Source: V$SQL. **Reading:** bind-sensitive or bind-aware children mean adaptive cursor sharing is active; plan differences may then be intentional.

Recorded reasons for multiple child cursors (V$SQL_SHARED_CURSOR)

Shown when more than one child exists. Counts of children with: OPTIMIZER_MISMATCH (different optimizer settings), OPTIMIZER_MODE_MISMATCH, OUTLINE_MISMATCH, BIND_MISMATCH, BIND_EQUIV_FAILURE (adaptive cursor sharing created a new child), ROLL_INVALID_MISMATCH (invalidated, for example after a statistics gather). Only flags with a non-zero count are listed.

Related Table Statistics and Related Index Statistics

Shows	• **Tables**: owner, name, last analyzed, **Statistics Status**, NUM_ROWS, BLOCKS, SAMPLE_SIZE. • **Indexes**: owner, name, table, index type, last analyzed, Statistics Status, **Status / Validity**, NUM_ROWS, DISTINCT_KEYS, LEAF_BLOCKS.
Data tier	DICTIONARY (objects identified from the current plan in V$SQL_PLAN, with DBA_HIST_SQL_PLAN as fallback).
Source	DBA_TAB_STATISTICS, DBA_IND_STATISTICS, DBA_TABLES, DBA_INDEXES, DBA_IND_PARTITIONS, DBA_IND_SUBPARTITIONS (ALL_* if DBA_* is not readable).
Statuses	• **STALE**: Oracle flags STALE_STATS = YES. • **NOT STALE**: not flagged; age alone is not a defect. • **MISSING**: LAST_ANALYZED is NULL (no statistics gathered). • **UNKNOWN**: not visible to this user, or STALE_STATS is not populated (common for indexes), or index type LOB/DOMAIN. • **VALID / UNUSABLE / N/A (partitioned)** for indexes: UNUSABLE indexes (or partitions) are shown in red with a count.
Limits	With ALL_* views only accessible objects appear and others show UNKNOWN. An empty list means the plan could not be read or contains no tables, not that none are used.

Related Table and Index Fragmentation Analysis

Shows	• **Tables**: type / partitioning, segment MB, allocated blocks, extents, BLOCKS in statistics, Wasted Space %, and a classification with evidence. • **Indexes**: segment MB, allocated blocks, extents, Wasted Space %, BLEVEL and a classification with evidence.
Data tier	DICTIONARY (sizes) plus statistics. This is a **statistics-based indicator, not a measurement**. No VALIDATE STRUCTURE or segment analysis is run.
Source	DBA_SEGMENTS with the statistics views above.
Table method	Estimated block density = NUM_ROWS x AVG_ROW_LEN compared with BLOCKS x block size; Wasted Space % = 100 minus density.
Index method	Allocated blocks divided by LEAF_BLOCKS; Wasted Space % = 100 x (1 - 1 / ratio).
Classes	• **LOW / NO SIGNIFICANT ISSUE IDENTIFIED**: below frag_min_mb, or above the thresholds. • **MODERATE / REVIEW REQUIRED**: table density below frag_density, or index ratio above frag_ratio (only for segments of at least frag_min_mb). • **NOT APPLICABLE**: temporary table, index-organized table, or LOB / DOMAIN / CLUSTER index. • **NOT ASSESSED**: no DBA_SEGMENTS access, segment lookup failed, no segment, statistics missing or incomplete, compressed table, or statistics flagged STALE. • A HIGH label exists in the code but is **never assigned automatically** (stated in the header).
Limits	Pre-allocated extents raise the index ratio; the script says so. Sparse leaf blocks are not visible from the ratio. Confirm with a measurement before reorganising anything (the report suggests how, as text).

Last DDL and DML Information

Shows	Per table: created, last DDL time, days since last DDL, last DML recorded, INSERTS, UPDATES, DELETES and flags: 'DDL in last 7 days', 'Truncated since statistics', 'no DML recorded', 'DML data unavailable'.
Data tier	DICTIONARY
Source	DBA_OBJECTS and DBA_TAB_MODIFICATIONS (table-level row, or partition rows for partitioned tables).
How to read it	DML counters since the last statistics gather show how far the data has moved from the statistics. Recent DDL can invalidate cursors and change plans.
Limits	Counters are flushed periodically, so they can lag behind recent activity.

Optimizer-related parameters whose value changed in the last 7 days

Shows	Parameter, Previous Value, New Value and the AWR snapshot end time when the change was seen. If nothing changed: a green note.
Data tier	AWR
Source	DBA_HIST_PARAMETER compared snapshot to snapshot (LAG), restricted to parameters named optimizer*, plus cursor_sharing, db_file_multiblock_read_count, pga_aggregate_target, sga_target, memory_target, star_transformation_enabled, query_rewrite_enabled, parallel_degree_policy, workarea_size_policy, statistics_level, session_cached_cursors and result_cache_mode. Case-only differences are ignored.
Limits	Session-level changes (ALTER SESSION) are not captured by AWR. Only this instance's parameters are examined.

The related-object analysis runs in its own error handler: if a step fails the report shows the Oracle error and continues. Findings derived here (stale statistics, recent DDL, unusable indexes, full scans, and so on) are collected and printed in section 6.2.

Section 6: Findings and Recommendations

Section 6 turns the evidence collected earlier into a prioritised list. It starts by naming the analysed plan hash value and its source. Subsections 6.1 and 6.2 are printed only when a plan is available.

6.1 Measured workload of the analysed plan

Shows	A two-column table: source, executions, total elapsed, elapsed per execution, CPU / user I/O / concurrency / application (lock) / cluster / PL/SQL time (each with a percent of elapsed), buffer gets, physical reads and rows per execution, buffer gets per row, share of buffer gets served from cache, rows per fetch, parse calls per execution, cursor loads / invalidations.
Data tier	CACHE (V$SQLSTATS, cumulative since load) first; if absent, AWR (DBA_HIST_SQLSTAT, sum of snapshot deltas in the last 7 days).
How to read it	• Check the 'Source' row to know whether you are reading cumulative or 7-day figures. • The time components can overlap (PL/SQL time contains the CPU and waits of the code it runs), so they do not add up to 100%. • If no execution has completed, per-execution figures use one execution as divisor and should be read as totals.
Limits	If neither source has figures, workload-based findings are skipped; object, plan and parameter findings are still given.

6.2 Findings and recommendations

Shows	• A **Main finding** note (colour follows its priority, with the total number of findings and how many are High). • A table: #, Priority (High / Review / Info), Finding, **Evidence from this SQL_ID** (object names, numbers, timestamps, predicates, plan step ids, plan hash values) and **Recommended action**.
Data tier	Mixed. The evidence line states its own source.
How to read it	See chapter I for severity labels, the list of findings with their thresholds, and why one may not appear.
If no finding	A green note says no finding crossed its threshold, repeats the elapsed time per execution, and suggests the cause may be outside the statement (caller, network, application time, instance-level waits).

Anonymous PL/SQL block path

If the SQL_ID is an anonymous PL/SQL block, section 6 does not analyse a plan. Instead it lists up to 10 statements that ASH recorded under this block as top-level SQL: SQL_ID, ASH samples, operation and the start of the SQL text. Source: V$ACTIVE_SESSION_HISTORY, then DBA_HIST_ACTIVE_SESS_HISTORY if nothing is found. Run this script again for those SQL_IDs.

I. How to read the findings

I.1 Severity labels

Label	Colour	Meaning in the script
High	Red	Evidence points to a likely major contributor or a condition that deserves attention first (for example lock waits, cartesian join, UNUSABLE index, sessions blocked now).
Review	Amber	Worth investigating; the evidence may be real but its impact is not established.
Info	Blue	Context or low-priority observations.

Findings are sorted High, then Review, then Info; within a priority they keep the order in which the script generated them. The **Main finding** is the first one in that order. The priority of some findings depends on the evidence (the table in I.3 gives the rule).

I.2 Parts of a finding

- **Finding**: the title, usually naming an object or plan step.
- **Evidence from this SQL_ID**: the actual figures that triggered it. Always check them against the source section.
- **Recommended action**: guidance printed as text. Nothing is executed. Examples named in the report include gathering or unlocking statistics, creating extended statistics, loading a SQL plan baseline, rebuilding an index, testing a hint in a session, or measuring space with DBMS_SPACE. Test on a non-production system and follow your change control.
- **Confirmation text**: the script stores a 'how to confirm' sentence with each finding, but the findings table has no column for it, so it is not printed (chapter L). Use the confirmation ideas below instead.

Typical ways to confirm a change worked: re-run this report and compare the plan hash value and average elapsed time (sections 3 and 5); check that STALE_STATS reads NO or that LAST_ANALYZED is populated; check E-Rows against A-Rows in section 1.6; check that the wait or time share that triggered the finding has fallen.

I.3 Findings catalogue and thresholds

All thresholds below are taken from the code. Percentages are shares of the statement's elapsed time in section 6.1.

Workload and wait findings

Finding	Appears when	Priority
Elapsed time dominated by user I/O	User I/O wait is 40% or more of elapsed time	High if elapsed per execution is 5 s or more, else Review
Statement is CPU-bound	CPU is 60% or more of elapsed (and the I/O finding did not fire)	Same rule
Row-lock (application) waits	Application wait is 20% or more	High
Concurrency waits	Concurrency wait is 20% or more	Review
Cluster (RAC) waits	Cluster wait is 20% or more	Review
High logical I/O per row	1,000 or more buffer gets per returned row	Review
High-frequency statement	1,000+ executions, under 0.05 s each, at least 60 s total	Info
Cursor invalidated many times	5 or more invalidations	Review
Very small fetch size	2 or fewer rows per fetch and 100+ rows per execution	Info
Top historical wait	Top ASH event (7 days) is not CPU and takes 25% or more	Review
Sessions blocked right now	At least one session running this SQL is blocked	High

Plan and cursor findings

Finding	Appears when	Priority
Plan instability	More than one plan and slowest is 2 times or more the fastest	High if the analysed plan is the slowest, else Review
Several plans, similar performance	More than one plan but within 2 times	Info
Optimizer parameter changes	At least one change in the last 7 days (section 5.4)	Review
Cartesian join	A cartesian join step in the plan	High
Full table scan (objects not analysed)	Full scan present but table/index dictionary data could not be read	Info
Most work in one plan step	Needs row-source statistics; the step with the most self buffer gets	Review if it carries 50%+ of buffer gets and 100,000+ in total, else Info
Cardinality misestimate at a plan step	Estimate and actual differ 10 times or more at a step without an object (join, aggregation)	Review
Work area spills to temp	One-pass or multi-pass executions in V$SQL_WORKAREA	High if multi-pass, else Review

Object findings (from section 5.4)

Finding	Appears when	Priority
Stale optimizer statistics on a table	STALE_STATS = YES	High if the plan has a full scan of it or its row estimate is 10x+ off, else Review
No optimizer statistics on a table	LAST_ANALYZED is NULL (not a temporary table)	High
Partitions without statistics	Partitions or subpartitions with no statistics, and the table is neither stale nor missing	Review
Locked statistics while data changes	Statistics locked, not stale, DML at least 10% of NUM_ROWS	Review
Row estimate off by N times	10x or more on a table whose statistics are neither stale nor missing	High at 100x+, else Review
Recent DDL	LAST_DDL_TIME within 7 days	Review
Table truncated since statistics	Truncated flag set in DBA_TAB_MODIFICATIONS	Review
Chained or migrated rows	CHAIN_CNT is 10% or more of NUM_ROWS	Review
Possible table / index space inefficiency	Fragmentation class MODERATE	Info
Index UNUSABLE	Index or partitions with status UNUSABLE	High
No statistics on index	LAST_ANALYZED is NULL	Review
Index and table statistics disagree	Gathered more than 1 day apart and NUM_ROWS differ 2x or more	Review
Poor clustering factor	Table of 100,000+ rows and clustering factor at 50% or more of NUM_ROWS	Info
Full scan findings (up to 5 steps, highest cost first)	Full scan with no predicate; an INTERNAL_FUNCTION predicate (possible implicit conversion); a full scan although an index leads with a predicate column; or no index on the filtered column	High if selective and (100,000+ buffer gets or segment at least frag_min_mb); Review if large but not selective; else Info. The implicit-conversion finding is at least Review.

I.4 Why a finding appears or does not

- A finding appears only when its evidence crosses the threshold above. 'No finding' means no threshold was crossed, not that the statement is fine.
- Some findings need data that may be missing: no figures in V$SQLSTATS or the last 7 days of AWR (workload findings); no row-source statistics (hot-step and cardinality findings, shown as an information note); no privilege on a view (that finding is skipped).
- Some findings suppress each other by design: the CPU-bound finding is skipped if the I/O finding fires; the partitions-without-statistics finding applies only when the table is neither stale nor missing; the row-estimate finding applies only when statistics are neither stale nor missing.
- An aged-out cursor removes cache evidence (plan, workload, work areas). AWR may still supply workload and plan data.

J. Privacy and sharing cautions

- **Bind values are application data.** They can hold names, identifiers, amounts or other personal or confidential information.
- **Where they appear when show_binds = Y:** 1.5.1 (values used by monitored executions), 1.5.2 (captured values) and the peeked binds in 1.6.
- **Purpose of show_binds:** set it to N to leave bind values out, in particular when the report is e-mailed or shared outside the DBA team. The script states this.
- **E-mail copies everything.** The helper sends the whole HTML (body and attachment). With send_mail = Y as shipped, you are prompted for an address after every report.
- **Reader note:** show_binds = N does not remove everything sensitive. The SQL text (1.2), plan predicates (1.6), the SQL Monitor text report (1.5), user names, host name, schema and object names are still in the report. Review the HTML before sharing it.
- The report file stays in the current directory until you delete it.

K. Troubleshooting and interpretation notes

Message or symptom	What it means	What to check
'Report not generated ... privileges that can change the database'	The privilege check found privileges from the list in E.2 and allow_privileged is not Y.	Use a SELECT-only account. Override only deliberately (chapter E).
'neither a numeric SID nor a 13-character SQL_ID'	Input rejected.	Re-run with a numeric SID or a 13-character SQL_ID.
'No SQL_ID could be resolved'	The session does not exist, or has neither a current nor a previous SQL_ID.	Confirm the SID in V$SESSION; try the SQL_ID instead.
'AWR views ... are not readable'	No SELECT on DBA_HIST_SQLSTAT / DBA_HIST_SNAPSHOT.	Grant access (and confirm licensing) or accept memory-only data.
'This SQL_ID is an anonymous PL/SQL block'	Command type 47 has no plan of its own.	Analyse the SQL_IDs listed in section 6.
'No SQL Monitor entries exist'	Statement not eligible, or entry aged out, or STATISTICS_LEVEL too low.	See limits of 1.4.
'CONTROL_MANAGEMENT_PACK_ACCESS is ...'	Tuning Pack disabled by configuration; a licensing/configuration setting, not a script error.	Leave as is, or enable only if licensed.
'cursor is no longer in the shared pool'	Aged out or flushed.	Use AWR parts (1.7, 3.1, 4.4, 6.1 fall back to AWR).
'No AWR plan history' / 'AWR is not accessible'	AWR holds nothing for the statement, or access is missing.	Check privileges; very short or rare statements may not be captured.
'No plan outline was returned'	The stored plan carries no outline data, or the release does not report it.	Informational.
'... could not be retrieved: ORA-...' (yellow)	A subsection failed; the rest continues.	Fix the cited privilege or object, then re-run.

Message or symptom	What it means	What to check
Tables show UNKNOWN or NOT ASSESSED	Statistics, DBA_SEGMENTS or ownership not visible; or STALE_STATS not populated.	Grant DBA_* views listed in chapter G.
'hot-step and cardinality checks were skipped'	No actual row counts for this cursor.	Collect row-source statistics (STATISTICS_LEVEL = ALL or the GATHER_PLAN_STATISTICS hint) in a test run.
Output ends with 'output truncated after N lines'	SQL text, SQL Monitor report or plan text longer than the display limit (400 / 3,000 lines for text and monitor; 5,000 lines default).	Query the full text directly if you need it.
'The report generator stopped on an unexpected error'	An error outside any subsection's handler.	Note the Oracle error and re-run; contact the script owner.
'E-mail could not be sent: ...'	The helper could not reach or log in to the SMTP server.	Check the SMTP_* environment variables; the report file is already written.
Plan in 1.7 differs from the 'current' plan in sections 5 and 6	1.7 uses the latest AWR snapshot; section 5 prefers the most recently active cached plan.	Compare both plan hash values in 5.3.

Other interpretation points

- Cursor-cache counters accumulate since the cursor was loaded; AWR figures cover separate snapshot intervals. Do not add them.
- An empty 'current' section (1.1, 1.3, 4.1, 4.2) only means nothing is happening at the moment the report ran. Run it while the problem is occurring for live data.
- SQL_IDs that differ only in literals appear separately; use section 2.2 to see them together.
- ASH-based time (section 4.4) is an estimate: samples x 10 seconds.
- Findings are guidance. Test any suggested change on a non-production system first.

L. Source notes and discrepancies

The following points were found by reading the SQL file and the mail helper. They are reported as observations. This README does not decide which side is correct.

#	Observation	Where in the source	Why it matters
1	Version labels differ: file name v1.2.8.3, banner 'version 2', run instruction and mail helper mention sqlid_diagnostic_v2.sql, change notes use V1.2, V1.2.7, V1.2.8 and V2. No note describes a '.3' revision.	File name; header banner; USAGE lines; sqlid_mail.py docstring	The invocation example does not match the attached file name. It is unclear which label is newest.
2	Change notes are not in version order: FIX (1.4), V1.2.8, V1.2.7, V2, V1.2 (5.4), V1.2 (6), then read-only guarantees. The FIX note sits under the 'version 2' banner.	Header comment	Ambiguous chronology; this README keeps the file order.
3	**allow_privileged**: comments say default N; the file assigns Y.	DEFINE section and its comment	As shipped, the privilege stop is disabled.
4	**send_mail**: comments say default N and 'E-mail is OFF unless send_mail = Y'; the file assigns Y.	DEFINE section and its comment; E-mail step at the end	As shipped, the e-mail helper runs after every report and prompts for an address.
5	**show_binds**: comment default Y matches the assigned Y.	DEFINE section	No discrepancy; listed for completeness.
6	The comment says allow_privileged = Y gives 'a warning'. No warning is printed in the code reviewed when privileges are found and the override is Y.	DEFINE comment; opening block of the PL/SQL	A user may assume a warning will appear.

#	Observation	Where in the source	Why it matters
7	The header says all V$ / DBA_HIST queries are dynamic SQL so a missing privilege never breaks compilation. The cursor c_waits (used in 4.3) is static SQL on V$SESSION.	Header V2 note; DECLARE section	A missing SELECT on V$SESSION could prevent the block from compiling. V$SESSION is also needed by almost every part.
8	Each finding stores a 'how to confirm' sentence, and the V1.2 note says findings carry it, but the findings table has no such column and the text is not printed.	Header V1.2 (section 6) note; section 6.2 table	Confirmation guidance is only partly present, inside some recommendations.
9	A plan-instability recommendation tells you to compare 'the peeked binds in section 4'. Bind information is in 1.5.1, 1.5.2 and 1.6; section 4 contains waits and blocking.	Section 6 plan-instability finding text	A wrong cross-reference.
10	The related table / index statistics and fragmentation analysis are called 'section 5.4' in the notes. In the report they are unnumbered bold blocks inside 5.4 (Correlation with performance and environment).	Header V1.2 notes; section 5.4	No numbers for these blocks; this README names them by their bold labels.
11	The 'required privileges' list is shorter than the objects the code reads: V$SQL_PLAN_MONITOR, V$ACTIVE_SESSION_HISTORY, SESSION_PRIVS, ALL_TAB_COLUMNS are not listed. Other access is only in the change notes.	Header PRIVILEGES REQUIRED vs code	Use chapter G.3 for the extra objects.
12	The header says grants must be direct 'because of PL/SQL'. This README repeats it and has not verified it.	Header	Unverified claim from the source.
13	Licensing is checked only in 1.4 (CONTROL_MANAGEMENT_PACK_ACCESS). AWR, ASH and DBMS_SQLTUNE are used whenever the privilege exists.	Section 1.4 vs 1.5, 3.x, 4.4	The script does not enforce licence limits for those parts.
14	1.7 picks the plan of the latest AWR snapshot; sections 5 and 6 use the most recently active cached plan (AWR only as fallback).	Sections 1.7, 5	The two 'current' plans can differ.
15	A HIGH fragmentation label exists in the code but is never assigned automatically (the header says so).	Header V1.2; cls_badge	Not a discrepancy; explained to avoid confusion.